Тема: Valve - платформа автономной FinOps-оптимизации ресурсов Kubernetes с eBPF-телеметрией рантайма и подключаемыми движками рекомендаций.

Проблема:

В промышленных кластерах Kubernetes управление ресурсами страдает от трех системных проблем.
1 - overprovisioning, разработчики систематически выделяют реквесты и лимиты CPU и памяти с запасом, и по статистике крупных кластеров фактическая утилизация составляет всего 5-40% от зарезервированных мощностей, а компании платят за простаивающие ресурсы.
2 - underprovisioning, тут обратная ситуация. Когда лимиты занижены, и приложение убивается OOM Killer'ом или растут задержки из-за CPU Throttling, особенно при резких всплесках нагрузки.
3 - Дрейф профиля нагрузки, потребление сервиса меняется во времени, причём в обе стороны. Сервис мог годами работать тихо, а после выката новой фичи начал потреблять в разы больше. А мог наоборот: раньше был тяжёлым, потом оптимизировали код или снизили нагрузку, и ресурсы остались выделены с большим запасом. Статические лимиты в репозиториях такие изменения не отражают.

При этом традиционный мониторинг (cAdvisor + Prometheus) опрашивает cgroups снаружи контейнера раз в 10-15 секунд и не успевает фиксировать короткие пики нагрузки, которые длятся доли секунды. Приложение за миллисекунды аллоцирует память, лимит превышается, ядро срабатывает OOM Killer, процесс убивается, под получает статус OOMKilled, а в Grafana график ровный, потому что пик попал между замерами.

Дополнительно существует проблема на уровне процессов: лимиты жестко прописаны в YAML-манифестах и Helm-чартах внутри репозиториев каждого микросервиса. Если инженер нашёл кривые лимиты у 50 сервисов, ему нужно сделать 50 MR в 50 разных репозиториев, найти разработчиков, дождаться ревью и дотащить это до прода, а это недели бюрократии.

Проект состоит из Control Plane - плоскость управления и применения и Decision Plane - плоскость рекомендаций:

Control Plane - отвечает за то, что бы рекомендации превращались в реальные изменения в кластере:

1 - eBPF-агент в виде DaemonSet на каждой ноде, без вмешательства в код приложения, собирает телеметрию рантайма приложения на Go изнутри ядра Linux.
2 - Хранение исторических метрик в PostgreSQL для анализа профиля нагрузки во времени.
3 - Mutating Admission Webhook для новых подов подставляет исторически оптимальные реквесты и лимиты на старте.
4 - Controller на Go принимает решение о применении рекомендаций и взаимодействует с kube-apiserver.
5 - Применение изменений. Обычно лимиты меняются на лету, без перезапуска пода. Но так можно не всегда: например, если лимит памяти опускается ниже текущего потребления, или меняется QoS-класс, или сама политика требует явного применения. В таких случаях Valve делает управляемый перезапуск пода, и всё это видно в UI, что меняем, почему, кого затронет.
6 - Policy Engine: правила, по которым Valve решает, что можно менять, а что нет. Сюда входят:

- locks - ручная фиксация лимитов для важных сервисов (например, базы данных). Поставил замок, и автоматика этот сервис не трогает.
- exceptions - исключения для сервисов, которым нужны особые условия. Например, сервису нужен лимит в 10 раз выше реквеста, и это нормально. Добавили в исключение и всё.
- cooldown - пауза между изменениями. Чтобы не дёргать лимиты каждую минуту.
- step threshold - минимальный шаг изменения. Если разница 1-2 единицы, ничего не меняем. Иначе поды будут постоянно перезапускаться.
- canary - сначала применяем на одном поде, смотрим, всё ли ок, потом на остальных.
- min/max - нижняя и верхняя граница. Чтобы автоматика не поставила лимит ниже плинтуса или выше разумного.
  7 - Audit log. Valve пишет, что изменил, когда, почему и кого затронуло. Чтобы всегда можно было посмотреть историю и понять, откуда взялись текущие лимиты.

Decision Plane - плоскость рекомендаций, отвечает за то, какие лимиты должны быть:
1 - Движки можно подключать. Valve не считает лимиты по одной жёсткой формуле. Вместо этого он даёт возможность подключать разные движки через единый интерфейс. Можно брать наш базовый, или писать свой.
2 - ValvePredictive, собственный движок Valve. Считает лимиты на основе телеметрии рантайма Go, которую собрал eBPF-агент (аллокации в куче, активность горутин, паузы GC, троттлинг CPU).
3 - KRR Adapter, обёртка над Robusta KRR. Уже готовый open-source инструмент, который считает рекомендации по историческим метрикам из Prometheus. Подключается как один из движков.

4 - Percentile Engine, простой движок на перцентилях. Берёт P50, P95, P99 потребления и на их основе предлагает лимиты. Нужен как базовый вариант для сравнения.
5 - Static Rules, ручные правила. Можно задать вручную, для такого-то сервиса такие-то лимиты. Без всякой математики.
6 - Custom Engines, свои движки. Люди могут написать движок на любом языке и подключить его через API. Например, если у команды уже есть своя математика расчёта лимитов.
7 - UI. Через интерфейс можно управлять правилами, исключениями, замками и движками. Смотреть историю изменений, сравнивать рекомендации разных движков и видеть, кто что предлагает.
8 - Понятность. Каждый движок отдаёт не просто числа, а ещё и объясняет, почему он предлагает именно такой лимит, насколько он в этом уверен, и нужен ли для применения перезапуск пода.

Научная новизна:

Научная новизна работы складывается из четырёх пунктов.
1 - Переход от реактивного мониторинга к проактивному. Традиционные системы (cAdvisor, Metrics Server) смотрят на контейнер снаружи и работают с уже случившимися фактами: сколько CPU съел, сколько памяти занял. Valve смотрит внутрь рантайма Go и использует внутренние показатели как предсказатели потребления. Не сколько уже съел, а сколько собирается съесть, частота и объём аллокаций в куче, активность планировщика горутин, паузы и задержки сборщика мусора. Это позволяет реагировать на изменения до того, как они превратились в инцидент.

2 - Архитектура подключаемых движков рекомендаций. Valve не считает лимиты одной жёсткой формулой. Он даёт подключать разные движки через единый интерфейс: собственный на eBPF-признаках, KRR от Robusta, простой на перцентилях, ручные правила, сторонние реализации через API. Это даёт две вещи: можно сравнивать разные подходы на одном кластере в одинаковых условиях и можно расширять систему без переписывания ядра.

3 - Безопасный автономный контур управления. Автоматически менять лимиты в проде без предохранителей это опасно. Поэтому в Valve отдельным слоем идут правила: замки для критичных сервисов, исключения, пауза между изменениями. Плюс минимальный шаг изменения, канареечное применение и границы min/max. Это превращает оптимизатор из чёрного ящика, который что-то там меняет, в управляемый контур с гарантиями и предсказуемым поведением.

4 - Честное применение изменений. В научной работе важно не обещать магию. Valve не заявляет, что всегда может менять лимиты без перезапуска. Он использует In-place Resize там, где это реально возможно, и переходит к управляемому перезапуску там, где нельзя. Оба сценария видны в UI: что меняем, почему, кого затронет. Это делает систему применимой на реальном продакшене, а не только в лабораторных условиях.

Стек:

Kernel space

- Язык C, подсистема eBPF (clang/LLVM, загрузка через sys_bpf)
- Точки перехвата: kprobes (sys_enter_mmap), uprobes (runtime.mallocgc), tracepoints планировщика (sched_switch)
- Доступ к task_struct и cgroup_id для привязки метрик к контейнеру
- Обмен с user space через BPF Ring Buffer

User space

- Go, библиотека cilium/ebpf для чтения BPF-карт
- client-go и controller-runtime для работы с Kubernetes API
- DaemonSet для развёртывания агента на каждой ноде
- Mutating Admission Webhook для стартовой подстановки лимитов
- Controller для применения рекомендаций

Decision Plane

- Сервис движков рекомендаций на Go с единым интерфейсом для подключения своих движков
- Встроенные движки: ValvePredictive (на eBPF-признаках), KRR Adapter, Percentile Engine, Static Rules

Policy Engine

- Внутри Controller, на Go
- Замки, исключения, cooldown, step threshold, canary, min/max

UI (Valve Control Center)

- Wails: бэкенд на Go, фронтенд на Vue3 с TypeScript

Хранение данных

- PostgreSQL - исторические метрики, audit log, рекомендации

Ожидаемый результат (важно, это только MVP):

- Снижение оверпровижининга: автоматическое сжатие неиспользуемых лимитов, за которые компании платят впустую.
- Предотвращение OOMKilled и CPU Throttling: своевременное расширение лимитов до того, как под упадёт или начнёт тормозить.
- Экономический эффект для бизнеса: меньше денег уходит на простаивающие ресурсы.
- Автоматизация рутины: отказ от массовых Merge Request по правке лимитов в репозиториях сервисов.
- Управляемое применение изменений: видно, что меняем, почему и кого затронет. Есть замки, исключения и откат.
- Возможность подключать разные движки рекомендаций и сравнивать их на одном кластере.
- Проверка на реальном кластере: сравнение с cAdvisor и KRR, оценка точности и влияния на SLO